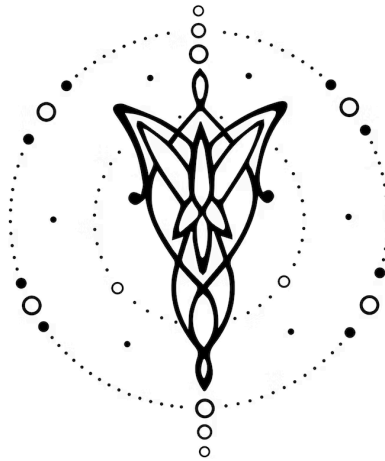




Informe Stage III: Backend

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

13 de junio de 2026

1. Equipo

Nombre	Apellido	Legajo	E-mail
Francisco	Trivelloni	64141	ftrivelloni@itba.edu.ar
Lorenzo	Pizzuto Beltran	64123	lpizzutobeltran@itba.edu.ar

Tabla de Contenidos

1. Introduccion
2. Modelo Computacional
 - 2.1. Dominio
 - 2.2. Lenguaje
3. Implementacion
 - 3.1. Frontend
 - 3.2. Backend
 - 3.3. Adicionales
 - 3.4. Dificultades Encontradas
4. Futuras Extensiones
5. Conclusiones
6. Referencias
7. Bibliografia

1. Introduccion

MipASM es un compilador para un lenguaje de programación de estilo C orientado a composición musical. Su entrada es un archivo .mip y su salida principal es un archivo Standard MIDI File (.mid) que puede reproducirse en reproductores, estaciones de audio digital o sintetizadores compatibles con General MIDI.

El objetivo de la etapa III fue completar el recorrido del compilador desde el AST construido por el frontend hasta la generación del artefacto final. Para ello se incorporaron dos responsabilidades centrales: el análisis semántico del programa y la generación de código específico del dominio musical. En este proyecto, la generación no produce código de máquina ni un programa ejecutable, sino una representación temporizada de eventos musicales que luego se codifica como MIDI formato 1.

La arquitectura final queda organizada de la siguiente manera: lectura de entrada y expansión de bibliotecas, análisis léxico, análisis sintáctico, construcción del AST, análisis semántico, interpretación del AST en tiempo de compilación y emisión binaria del archivo MIDI. El resultado es la escritura de bytes de un archivo MIDI.

2. Modelo Computacional

2.1. Dominio

El dominio elegido es la composición musical programática. El programador describe pistas, notas, silencios, instrumentos, tempo, compás, volumen y otros elementos del midi usando un lenguaje estilo C. El compilador transforma esas instrucciones en eventos MIDI ordenados temporalmente.

El modelo temporal del lenguaje usa beats como unidad principal. Un beat equivale a una negra y se convierte internamente a 480 ticks MIDI. La ejecución conceptual del programa mantiene un cursor de tiempo global: una instrucción play agrega una nota en el tiempo actual y avanza el cursor según su duración; una instrucción rest avanza el cursor sin emitir sonido; las asignaciones, declaraciones y cambios de control no consumen tiempo.

Para expresar simultaneidad se usa `sync { ... }`. Dentro de ese bloque, cada sentencia de primer nivel comienza en el mismo beat inicial, y al terminar el bloque el cursor global avanza hasta el final de la sentencia más larga. Esto permite construir acordes o capas paralelas sin introducir un sistema de concurrencia real.

El artefacto final es un archivo MIDI Standard MIDI File formato 1. La implementación escribe un track conductor para metadatos globales como tempo y compás, más un track MIDI por cada pista declarada con `init_track`. Las pistas se asocian a canales MIDI, y cada evento de dominio se traduce a mensajes MIDI: Note On, Note Off, Program Change, Control Change o meta eventos.

2.2. Lenguaje

MipASM adopta una sintaxis cercana a C para reducir la fricción de uso: declaraciones, asignaciones, bloques con llaves, if, for, while, expresiones aritméticas, relacionales y lógicas, y una función obligatoria `void main()`.

Los tipos soportados son:

- `int`: valores enteros para notas, canales, contadores y constantes MIDI.
- `float`: valores numéricos con decimales.
- `boolean`: condiciones y valores lógicos representables desde enteros.
- `duration`: duraciones musicales en beats.
- `track`: referencia a una pista MIDI creada con `init_track`.

El lenguaje incorpora operaciones del dominio:

- `track name = init_track(channel);`
- `play(track, note, duration);`
- `rest(track, duration);`
- `sync { ... }`
- `set_tempo(bpm);`
- `set_time_signature(numerator, denominator);`
- `set_instrument(track, program);`
- `set_volume`, `set_pan`, `set_attack`, `set_sustain`, `set_modulation` y `set_reverb`.

También se proveen bibliotecas estándar en `lib/stdlib`, por ejemplo `audio`, `notes`, `instruments`, `drums`, `scales` y `chords`. Estas bibliotecas permiten escribir programas más expresivos mediante constantes como notas, duraciones, instrumentos General MIDI y dinámicas.

3. Implementacion

3.1. Frontend

El frontend está implementado con Flex y Bison. Flex reconoce palabras clave, identificadores, literales, operadores, delimitadores, comentarios e includes. Bison define la gramática del lenguaje y construye un AST mediante acciones semánticas.

La entrada al compilador se maneja desde `src/main/c/EntryPoint.c`. Allí se pasean opciones de línea de comando como `-o`, `--help` y `--version`, se deriva la ruta de salida cuando no se especifica, y se inicializan los módulos de compilación. El flujo principal ejecuta el análisis sintáctico, luego el análisis semántico y finalmente la generación.

El scanner está definido en `src/main/c/frontend/lexical-analysis/FlexPatterns.l`. Una decisión relevante es que los includes bien formados se consumen en el lexer: incluye "<stdlib/audio>" no llega al parser como una sentencia común, sino que Flex resuelve el archivo con `LibraryLocator`, apila un nuevo buffer de entrada y empalma los tokens de la biblioteca en la compilación actual. Esto permite que las bibliotecas sean archivos MipASM ordinarios con declaraciones globales.

La gramática principal se encuentra en `src/main/c/frontend/syntactic-analysis/BisonGrammar.y`. El programa aceptado tiene directivas, declaraciones globales y exactamente una función `void main()`. Las declaraciones globales admiten variables y tracks, mientras que las instrucciones musicales viven dentro de bloques de sentencias. La gramática declara precedencias para mantener el comportamiento esperado de expresiones aritméticas, relacionales y lógicas.

El AST está definido en `src/main/c/frontend/syntactic-analysis/AbstractSyntaxTree.h`. Incluye nodos para programa, includes, main, bloques, declaraciones, inicialización de tracks, asignaciones, operaciones musicales, control de flujo y expresiones. Esta representación es la frontera entre frontend y backend: a partir de ella ya no se trabaja con tokens, sino con una estructura explícita del programa.

3.2. Backend

El backend se divide en dos fases: análisis semántico y generación de código.

El análisis semántico está implementado en `src/main/c/backend/semantic-analysis/SemanticAnalyzer.c`. Recorre el AST y mantiene una tabla de símbolos con scopes anidados. Válida declaraciones duplicadas en el mismo scope, uso de identificadores declarados, asignaciones a constantes, compatibilidad de tipos, condiciones booleanas, argumentos de operaciones musicales, uso correcto de tracks y reglas específicas como que `set_time_signature` reciba enteros.

El sistema de tipos distingue `int`, `float`, `boolean`, `duration` y `track`. Se permite conversiones implícitas acotadas: `int` a `float`, `int` o `float` a `duration`, e `int` a `boolean`. En cambio, un `track` no puede usarse como número, y una expresión no compatible es rechazada antes de llegar a generación.

La generación está implementada en `src/main/c/backend/code-generation/Generator.c`. Esta fase funciona como un intérprete de tiempo de compilación: ejecuta las declaraciones y sentencias del AST, evalúa expresiones, actualiza variables, recorre loops, toma ramas de `if`, mantiene scopes de ejecución y acumula eventos musicales en una estructura `MusicProgram`.

Este enfoque es adecuado para el dominio porque el resultado buscado es una pieza musical completamente determinada al compilar. No hay runtime de ejecución posterior: los loops se “desenrollan” durante la generación, las expresiones se pliegan al evaluarse, y solo quedan eventos temporizados para emitir en el archivo MIDI.

Durante la generación también se aplican validaciones dependientes de valores concretos. Por ejemplo, `set_tempo` exige BPM positivo, `set_time_signature` exige numerador entre 1 y 255 y denominador potencia de dos entre 1 y 32, y los loops tienen un límite de 100000 iteraciones para evitar compilaciones infinitas. También se detectan variables no inicializadas cuando afectan playback, duraciones, canales o límites de loops. El análisis semántico (`SemanticAnalyzer`) hace los chequeos estáticos. La fase de generación, hace las validaciones que dependen de valores concretos y que recién se conocen al evaluar.

La emisión binaria MIDI vive en `src/main/c/backend/code-generation/MidiEmitter.c`. Este módulo es el único que conoce los bytes del formato MIDI. Recibe eventos de alto nivel y escribe:

- Header MThd con formato 1, cantidad de tracks y division PPQ.
- Track conductor con nombre, tempo por defecto de 120 BPM si no se define uno inicial, eventos de tempo y compás.
- Un MTrk por cada track declarado, con nombre de pista, eventos de canal y fin de pista.

Los eventos se convierten a ticks con 480 ticks por beat, se ordenan por tiempo absoluto, se codifican como deltas MIDI y se escriben con cantidades variables donde corresponde. Las notas generan pares Note On y Note Off. Los controladores se mapean a Control Change, por ejemplo volumen al controlador 7, pan al 10, sustain al 64, modulación al 1 y reverb al 91. Los instrumentos se emiten como Program Change.

3.3. Adicionales

El proyecto incluye una biblioteca estándar en `lib/stdlib` con constantes musicales reutilizables. Esto mejora la ergonomía del lenguaje porque evita que el usuario memorice numeros MIDI para notas, instrumentos, dinamicas o canales especiales como percusión.

También se implementó una interfaz de línea de comando más completa: lectura desde archivo o stdin, salida con `-o`, derivacion automatica de `.mip` a `.mid`, `--help`, `--version` y códigos de salida consistentes.

La documentación del repositorio incluye una guia de uso (`doc/PROGRAMMING_GUIDE.md`) y una referencia de features (`doc/LANGUAGE_FEATURES.md`). Esas guias describen tanto el lenguaje como la relacion entre cada sentencia musical y su implementacion en Flex, Bison, AST, semántica, generación y emisión MIDI.

La documentación puede ser leída por LLMs para poder escribir código MIPASM y generar música bastante mala.

La validación automatizada se organiza con tests de aceptación y rechazo en `test/c`. Al momento de este informe hay 31 casos de aceptación y 28 casos de rechazo. El script `src/main/bash/test.sh` compila cada caso esperado como válido hacia `/dev/null`, verifica que los casos inválidos fallen, y también prueba escenarios de CLI.

3.4. Dificultades Encontradas

Una dificultad central fue separar correctamente las responsabilidades entre semántica y generación. Algunas reglas pueden validarse solo con tipos y símbolos, por ejemplo que un identificador exista o que una condición sea boolean-compatible. Otras dependen del valor concreto en tiempo de compilación, como tempo mayor que cero, denominador de compás potencia de dos, variables no inicializadas o loops que no terminan. La solución fue mantener un analizador semántico estático y una fase de generación que también reporta errores cuando evalúa valores reales.

La emisión MIDI también requirió cuidado. El formato usa campos big-endian, cantidades variables para deltas, tracks con longitudes prefijadas y eventos que deben ordenarse. Para evitar escribir longitudes incorrectas, el emisor arma cada track en un buffer antes de emitir su chunk MTrk.

Los includes de biblioteca presentan otra complejidad: deben comportarse como si el código incluido estuviera escrito en el archivo fuente, pero sin cargar repetidamente una misma biblioteca. La solución usa el stack de buffers de Flex, resolución mediante LibraryLocator e include-once por ruta canónica.

4. Futuras Extensiones

Una extensión natural sería mejorar el diagnóstico de errores con ubicaciones más precisas, especialmente para errores dentro de bibliotecas incluidas. Actualmente la infraestructura de ubicaciones existe, pero el reporte podría enriquecerse con nombre de archivo, línea y columna de origen.

En cuanto al backend, se podría emitir información MIDI adicional como key signature, lyrics o markers, y agregar una abstracción de secciones (que agrupen eventos con su propio tempo/compás) — los cambios de tempo y de compás a lo largo de la pieza ya están soportados, pero no existe el concepto de sección. También se podría agregar exportación alternativa, por ejemplo a MusicXML.

5. Conclusiones

La etapa III completa el compilador MipASM al conectar el AST con una salida concreta del dominio. El análisis semántico garantiza que el programa sea coherente respecto de símbolos, tipos, scopes y uso de operaciones musicales. La generación interpreta el programa en tiempo de compilación y produce una lista de eventos temporizados. Finalmente, el emisor MIDI convierte esa representación en un archivo binario reproducible. Poder escuchar el primer archivo que compilo con éxito fue bastante gratificante.

El diseño elegido aprovecha que una composición MipASM es determinista al compilar. Por eso no se requiere runtime propio: el compilador ejecuta la lógica necesaria una sola vez y entrega el

artefacto final. Esta decisión simplifica la distribución y hace que el resultado sea portable, ya que cualquier entorno capaz de reproducir MIDI puede consumir la salida.

6. Referencias

- README.md: descripción general del proyecto, comandos de build, run, test e instalación.
- doc/PROGRAMMING_GUIDE.md: guía de programación del lenguaje MipASM.
- doc/LANGUAGE_FEATURES.md: referencia de features, operaciones musicales y mapeo de implementación.
- src/main/c/EntryPoint.c: flujo principal de compilación y CLI.
- src/main/c/frontend/lexical-analysis/FlexPatterns.l: definición léxica.
- src/main/c/frontend/syntactic-analysis/BisonGrammar.y: gramática del lenguaje.
- src/main/c/frontend/syntactic-analysis/AbstractSyntaxTree.h: estructura del AST.
- src/main/c/backend/semantic-analysis/SemanticAnalyzer.c: análisis semántico.
- src/main/c/backend/code-generation/Generator.c: interpretación del AST y generación de eventos.
- src/main/c/backend/code-generation/MidiEmitter.c: emisión del archivo MIDI.
- test/c/accept y test/c/reject: suite de pruebas de aceptación y rechazo.

7. Bibliografía

- Información en línea de el binario midi y las clases.